

ClawCraft

AI-Assisted 3D Art Pipeline

Complete Project Context

March 31, 2026

Acacia Williams (Xaynia)

CART 498 Generative AI | Concordia University | Solo Project

Mac ARM64 (Apple Silicon)

What ClawCraft Is

ClawCraft is an AI-assisted 3D art project. Users describe scenes in natural language via Discord, an AI agent (@ClawBot) generates Blender Python code, executes it inside Blender, renders the scene with EEVEE, and posts the resulting image back to Discord. The user evaluates the render, provides feedback, and the loop repeats. Every render is automatically logged with the full prompt, every Blender command executed, timing, outcome classification, and a workflow observation written by the LLM about the process itself.

The art explores childhood game worlds colliding with mundane adult spaces. A fridge that opens onto a blocky Minecraft-like landscape. Kitchen shelves holding game-world geometry next to coffee mugs. A hallway where objects gradually degrade from photorealistic to N64-era low-poly as you look deeper. The aesthetic contrast between photorealistic domestic environments and low-poly game elements is the core concept.

Both successes and failures are part of the work. The pipeline is the instrument; the art is the deliverable. The academic contribution is documenting what the AI-assisted workflow enables and limits compared to traditional 3D modeling.

Builds on a prior CART 498 project demoing OpenClaw itself (Discord bot for events, weather, image generation; 18-slide presentation delivered March 12).

Core Pipeline

```
User (Discord #blender, @ClawBot mention)
|
v
OpenClaw Gateway (Docker, ws://0.0.0.0:18789)
|
v
GPT-5.3-Codex (ChatGPT sub - free text)
| generates Blender Python (bpy) code
v
Blender Plugin (TypeScript, TCP socket)
| host.docker.internal:9876
v
Blender 5.1 (Mac host, headless EEVEE)
| renders scene
v
PNG to shared volume (/tmp/clawcraft)
|
v
MEDIA: directive posts image to Discord
|
v
Bot asks for feedback -> loop repeats
```

No MCP intermediary needed. The custom OpenClaw plugin talks directly to the Blender addon socket. The LLM generates Python code using Blender's bpy API, sends it over TCP, Blender executes it and renders with EEVEE, the PNG is base64-encoded back through the socket, decoded by the plugin, saved to a shared Docker volume, and posted to Discord via MEDIA: directive.

Model Configuration

Primary: GPT-5.3-Codex

- Model ID: openai-codex/gpt-5.3-codex (alias: Codex)
- Auth: OAuth through ChatGPT subscription. No per-token text cost.
- Context: 266k tokens, text+image input
- Rate limits: 5-hour rolling window + daily cap. Bot stops if exhausted.

Backup: Claude Sonnet 4

- Model ID: anthropic/claude-sonnet-4-20250514 (alias: Sonnet)
- Auth: API key in container env (ANTHROPIC_API_KEY)
- Cost: ~\$3/M input, ~\$15/M output. Roughly \$0.01-0.06 per render.
- Status: Registered, NOT auto-fallback. Manual switch only to avoid silent billing.

Image Generation: gpt-image-1 (DALL-E)

- Accessed via openai-image-gen skill. Costs ~\$0.04-0.08/image.
- CRITICAL: Has NOT been used yet. Required as second ML architecture for rubric.

Switching Models

```
# Switch to Sonnet
docker exec openclaw-openclaw-gateway-1 node /app/dist/index.js \
  config set agents.defaults.model.primary \
  anthropic/claude-sonnet-4-20250514
docker exec openclaw-openclaw-gateway-1 sh -c \
  'echo "{}" > /home/node/.openclaw/sessions.json'
```

Skills & Plugins

Custom Plugin Tools (extensions/blender/)

- **blender** - Core tool. Actions: scene_info (query objects/materials), object_info (detail for named object), execute (run Blender Python, bpy pre-imported, 180s timeout), render (auto-logs to render_log.json + Notion, EEVEE 1920x1080), log_note (append note to render), rate (quality 1-5). Tool instructs bot to always ask for evaluation after render.
- **canvas** - HTML-to-image via headless Chromium (Playwright). CDN libs: Mermaid (diagrams), KaTeX (math), Shiki (code highlighting), Iconify (10k+ icons), D3.js (data viz), Chart.js (charts).
- **slidev** - Markdown to slide deck. Slidev build, Playwright screenshots each slide, posted to Discord. light-icons theme. Addons: python-runner, plotly.js.

Enabled Skills (10)

- **openai-image-gen** - DALL-E concept art (costs money per image)
- **github** - Full gh CLI access

- **nano-pdf + pypdf** - PDF editing/reading
- **video-frames** - ffmpeg frame extraction
- **discord, weather, healthcheck, skill-creator, asciinema** - Messaging, utilities, terminal recording

Python Packages

matplotlib, pillow, plotly, yt-dlp, nano-pdf, pypdf

Render Logging System

Every render automatically writes a structured entry to render_log.json AND creates a row in the Notion Renders database. Zero manual bookkeeping.

Fields captured per render:

- id (e.g., render_001)
- timestamp (ISO 8601)
- scene_id (e.g., fridge_portal)
- iteration (auto-incremented per scene)
- user_prompt (original request)
- refined_from (previous render ID if iteration > 1)
- refinement_notes (what changed, why)
- blender_commands (exact Python code executed)
- model_used
- render_success (boolean)
- error_message (if failed)
- render_time_seconds
- output_image (filename)
- outcome_type: "success" | "partial" | "failure" | "happy_accident"
- workflow_observation (one sentence about the PROCESS, not the image)
- quality_rating (1-5, set later)
- notes (free text, appended later)

Bot Commands

- /log-note render_001 "text" - append note to render entry
 - /rate render_001 4 - set quality rating 1-5
 - /session-summary - today's stats, outcome breakdown, top observation
 - /render-report - generate Slides deck from render_log.json
 - "run next prompt" - start next unstated scene from prompt_queue.md
-

Notion Integration (added March 26)

Page ID: 32f366192ab580269bc9d49f3827e498. Token and 4 DB IDs stored in ~/src/openclaw/.env (loaded by docker-compose).

- **Scenes DB** (32f36619-2ab5-8123-aedc-edeb780ff): 6 rows. Properties: Name, Status (Not Started/In Progress/Complete), Iterations, Best Render, Notes.
- **Renderers DB** (32f36619-2ab5-8149-af30-c540c854aff4): Auto-populated on every render. Relation to Scenes. Properties: Name, Scene, Iteration, Outcome, Render Time, Prompt, Workflow Observation, Quality Rating, Image URL, Date.
- **Tasks DB** (32f36619-2ab5-8166-82ec-f09730324655): 12 tasks for Week 2.
- **Milestones DB** (32f36619-2ab5-81cf-ab76-fc564d44fcf5): 30 milestones, Weeks 1-5. Week 1 all complete.

blender-tool.ts pushRenderToNotion() fires after every render (best-effort, non-blocking). Also updates Scene row Iterations count and sets Status to 'In Progress'. Uses Node fetch() to call Notion REST API directly.

Workspace Files

Located in container at /home/node/.openclaw/workspace/ (mounted from ~/.openclaw/workspace/ on Mac).

- **SOUL.md** - Bot personality, aesthetic, 6-step iterative loop, workflow observation instructions
- **TOOLS.md** - Environment reference, render settings, scene IDs, commands, CDN libraries
- **render_log.json** - Auto-populated per render (starts as empty [])
- **workflow_notes.md** - Personal scratchpad for process observations
- **prompt_queue.md** - 6 pre-planned scenes with status tracking checkboxes
- **milestones.md** - 5-week deliverable checklist
- **hand_built_log.json** - For manual comparison logging (likely unused)
- **HEARTBEAT.md** - DISABLED (generates noise)

6 Planned Scenes

Each explores the collision between photorealistic domestic spaces and low-poly game-world elements.

1. **fridge_portal** - Open fridge, blocky low-poly landscape inside extending to horizon. Voxel terrain, pixel trees, 8-bit sky. Kitchen lighting spills into game world.
2. **shelf_artifacts** - Kitchen shelf with normal objects alongside low-poly game objects (potion bottle, golden apple, pickaxe). Realistic lighting on both.
3. **bathtub_overflow** - Bathtub with low-poly water (flat blue planes). Voxel fish. Water becomes realistic on tile floor. Two rendering styles colliding.
4. **window_gameworld** - Normal room, window onto retro game landscape. Flat green ground, cube trees, pixelated sun. Realistic curtains. Lighting transition at windowsill.
5. **dining_dungeon** - Dinner table set normally. Under the table: dungeon entrance with stone blocks, torches, dark staircase. Kitchen light vs dungeon glow.

6. hallway_renderdist - Normal hallway. Objects degrade photorealistic to N64-era low-poly with distance. Fog at far wall. Gradual, not hard cutoff.
-

Key Paths

Mac (host)

- Blender 5.1: /Applications/Blender 3.app
- Blender MCP addon: ~/Library/Application Support/Blender/5.1/scripts/addons/blender_mcp_addon.py
- Plugin source: ~/.openclaw/extensions/blender/
- OpenClaw source + compose: ~/src/openclaw/
- Docker-compose .env: ~/src/openclaw/.env
- OpenClaw config: ~/.openclaw/openclaw.json
- Post-start script: ~/src/openclaw/docker-post-start.sh
- Render output: /tmp/clawcraft/
- ClawDock helpers: ~/.clawdock/clawdock-helpers.sh
- Logs archive: ~/clawcraft_logs/

Docker (container: openclaw-openclaw-gateway-1)

- Config: /home/node/.openclaw/openclaw.json
 - Workspace: /home/node/.openclaw/workspace/
 - Plugin: /home/node/.openclaw/extensions/blender/
 - Renders: /home/node/.openclaw/media/renders/
 - Sessions: /home/node/.openclaw/sessions.json
 - Image: openclaw:local (OPENCLAW_INSTALL_BROWSER=1)
 - Ports: 18789 (gateway), 18790 (bridge), 1455
-

Persistence Model

Survives container recreate (Mac-mounted)

- Plugin files (~/.openclaw/extensions/blender/)
- Workspace files (~/.openclaw/workspace/)
- Config (~/.openclaw/openclaw.json)
- Docker image (openclaw:local), docker-compose.yml, .env
- Render files (/tmp/clawcraft/)
- docker-post-start.sh

Needs reinstall (docker-post-start.sh, ~60 seconds)

- apt: gh, ffmpeg, python3-pip, asciinema, build-essential

- pip: matplotlib, pillow, yt-dlp, nano-pdf, plotly
 - npm: @slidev/cli, themes, addons, @notionhq/client
 - render-html.mjs (Playwright HTML-to-PNG script)
 - Plugin file ownership fix (chown node:node)
 - jiti cache clear (stale compiled TypeScript)
 - Log file creation (render_log.json, workflow_notes.md)
-

Infrastructure Failures (documented history)

Post-start script issues

1. Wrong .env file: Notion vars written to ~/.openclaw/.env instead of ~/src/openclaw/.env. Container couldn't read them. docker-compose env_file reads from compose directory, not mounted config.
2. Container recreate cascade: fixing .env required docker compose up which recreated container, wiping all runtime installs.
3. Duplicate env entries from double-writing. Required Python dedup script.
4. Plugin ownership: Mac uid 501 vs container uid 1000. Cryptic permission errors if forgotten.
5. jiti cache stale code: old compiled JS runs after editing TypeScript. Must clear /tmp/jiti/ manually.
6. SSL cert failure: Python 3.14 on Mac rejects HTTPS. Had to use curl for all Notion API calls.
7. zsh reserved variable ('status' is read-only). Script failed silently.
8. Notion API annotation format wrong multiple times (annotations is sibling of text, not inside it).

Blender socket reliability

1. Manual start every session: Blender sidebar N > BlenderMCP > Start Server. No auto-start.
2. Socket drops without warning. Connected one minute, ECONNREFUSED the next.
3. No health check or auto-reconnect in the plugin.
4. host.docker.internal NAT via Docker Desktop occasionally fails.

OpenClaw CLI issues

1. message send posts AS the bot, doesn't trigger AI. Need: agent --to --message --deliver.
2. Agent falls back to embedded mode on gateway failure (different tool availability).
3. sessions.json doesn't auto-update on model change. Must nuke manually.
4. Tool name conflict warning: 'canvas' from blender plugin (cosmetic).
5. Retry spiral: one failure cascaded to 232 requests, \$5.09, 11.1M tokens. No circuit breaker.

Bottom line: Zero renders completed through the full Discord pipeline as of March 31. All individual components work. Socket reliability is the blocker.

Academic Requirements

Project Type: Option A

Generative AI-driven art project using at least two different media types and architectures, with written report.

Deliverables

1. Proposal - March 19 (submitted, graded, 10%)
2. Presentation - April 9 in class, recorded (20%)
3. Report - April 16 online (20%)

Presentation (April 9)

- 12 minutes strict max (6 explanation + 6 demo)
- Audience: NOT familiar with GenAI/ML
- Show successes AND failures
- Submit PDF to Moodle

Report (April 16)

- PDF, 2000-3000 words (excl. abstract, references, appendices)
- Chicago author-date citations
- ACM HCI LaTeX template recommended
- No code screenshots - link to GitHub
- Must include demo video link + code repo link
- Audience: IS knowledgeable in ML/AI - explain technical detail

Report Structure (from syllabus)

1. Abstract (150-200 words)
2. Introduction: context, literature review, significance, objectives
3. Project Description: vision, technologies, ML tasks, infrastructure, data, system integration
4. Process: development stages, challenges, successes, learning
5. Future Work
6. References (Chicago author-date)
7. Appendix: demo video, code repo, deployed app links

Two Media Types Required

1. 3D models / rendered scenes / animations (Blender)
2. Images (DALL-E concept art + Blender renders)

Two ML Architectures Required

1. Transformer: Codex for NL to Blender Python generation
2. Diffusion/Autoregressive: gpt-image-1 for concept art - NOT YET USED

Professor Feedback (6 exact points from Vig)

1. "Misleading at first" - Say upfront it's an iterative feedback loop (prompt > evaluate > refine). First paragraph. Don't bury it.
2. "Objective needs revision" - Goal isn't just art pieces. It's comparing human vs AI workflows and documenting insights. The art is the output; the insight is the contribution.
3. "Artistic direction with prompts" - Show actual planned prompts tied to artistic vision. Specific scenes, not generic descriptions.
4. "Measurable milestones" - Concrete weekly deliverables, not vague exploration goals.
5. "Literature review" - Engage critically. Say what sources found and how it relates to this work.
6. "Concise" - Proposal was 5 pages, rubric said 3. Keep report within 2000-3000 words.

Core insight: The professor wants a reflection on what the ClawCraft workflow enables vs limits compared to manual 3D modeling. The comparison is the academic contribution. That's writing, not infrastructure.

Operational Implications

- Document the feedback loop, not just the output. The Discord conversation IS the data.
- Compare to hand-building. Note what was faster, better, worse, surprising.
- Plan prompts in advance (done - 6 scenes in prompt_queue.md).

References (from proposal)

- Ahuja 2025 (BlenderMCP - MCP enables two-way Blender control, geometry cleanup often needed)
- Lima 2025 (3D-Agent)
- Eliason 2026 (FelixAI / OpenClaw agent)
- Ho et al 2020 (DDPM / denoising diffusion)
- Vaswani et al 2017 (Attention Is All You Need)
- Ouyang et al 2022 (RLHF / InstructGPT)
- Yao et al 2022 (ReAct - reasoning + acting in LLMs)
- Bitdefender, Cisco, Microsoft 2026 (OpenClaw security advisories)

Current Status (March 31, 2026)

- Pipeline: all components built and individually tested
- Zero renders through full Discord chain (socket reliability blocker)
- No DALL-E concept art generated (required for rubric)
- No animations, demo video, or GitHub repo
- Notion page partially complete
- Art production behind schedule (should have started March 26)

Timeline

- Week 1 (Mar 19-25): COMPLETE - pipeline built
- Week 2 (Mar 26-Apr 1): BEHIND - blocked by socket reliability
- Week 3 (Apr 2-8): Art production, presentation, demo video
- Week 4 (Apr 9): Presentation day
- Week 5 (Apr 10-16): Write report, due April 16

Backup Option

If MCP pipeline too unstable: Tetris-like game inside fridge scene. Export AI-generated Blender assets to UE/Unity. Prior experience with full Blender-to-game-engine pipeline.